

CMCMIS_SIMPLIFIED

Computerized Maintenance & Calibration Management Information System

PHASE 0 → PHASE 6 FINAL INSIGHTS BACKBONE DOCUMENT

Locked engineering memory · senior developer reference · documentation handoff

LOCKED STATUS

Phase 0 → 6 is treated as sealed backbone for this project.
This document consolidates the uploaded source artifacts into one senior-engineering insight reference.
Use this as the revision-ready map before writing Phase 7+ code.

Field	Value
Document	CMCMIS Phase 0→6 Final Insights Backbone
Prepared For	Deep Sorathiya (DS)
Project Environment	ISRO SAC-like high-precision engineering / government / defence-grade context
Role of this Document	One Word DOCX reference capturing phase-wise insights, decisions, patterns, risks, and next-build doctrine
Primary Mode	Senior Software Developer Engineer + Senior Technical Documentation Engineer
Build Rule	JavaScript only · live MySQL · raw SQL repositories · zero mock data · audit-first
Generated	2026-05-18 14:17

Table of Contents

Part	Title	Purpose
Part I	Executive Lock Summary	What is done, why it matters, and the DS locked meaning
Part II	Source Backbone	Every uploaded document and what it contributes
Part III	Phase 0→6 Integrated Timeline	Phase-wise progress and engineering maturity
Part IV	Product & Domain Doctrine	CMCMIS as a stateful, auditable metrology workflow system
Part V	Architecture Doctrine	3-tier monolith, modules, layers, request traces, data flow
Part VI	Database & Schema Insights	Legacy preservation, migrations, tables, RBAC, idempotency
Part VII	Auth/RBAC Security Insights	5 roles, 40 permissions, JWT, cookies, audit, visibility
Part VIII	Equipment Module Insights	Audit-first schema mapping, list/form/transactions, shell redesign
Part IX	Job Requests + Job Cards Insights	Phase 6 slice, state machines, row scopes, doctrines
Part X	End-to-End Operational Walkthroughs	Real workflows from login to verification and audit
Part XI	Implementation Laws & Review Checklists	Rules future code must obey
Part XII	Phase 7+ Handoff	The next locked build path and risk register
Appendix	Glossary + Final Lock Manifest	Definitions and final memory snapshot

Part I — Executive Lock Summary

1.1 The one-line project definition

SYSTEM DEFINITION

CMCMIS is the system that knows where every instrument in the lab is, what state it is in, when it needs calibration, who is working on it, and what was done to it for as long as the instrument exists.

1.2 The domain in five words

REGISTER → CALIBRATE → MAINTAIN → REPAIR → RETIRE

Every module, table, state transition, PDF, and audit log exists to preserve this lifecycle. The system is not a simple CRUD app; it is an operational memory engine for a high-precision engineering laboratory.

1.3 What is now locked

Locked Area	Meaning
Product scope	Industry-grade CMCMIS for metrology, calibration, maintenance, repair, registration, equipment lifecycle tracking, job workflow, inquiry, dashboard, and future reports/procurement/scheduling.
Architecture	3-tier monolith: React 18 + Vite + Tailwind frontend, Node.js + Express backend, MySQL 8.x persistence. On-premises/internal deployment.
Code style	JavaScript only. Backend CommonJS .js; frontend ESM .jsx/.js. No TypeScript in v1.
Database posture	Existing legacy database is preserved. Changes are additive/idempotent; no destructive drops/renames in sealed phases.
Auth model	Employee-ID login, bcrypt password hashes, JWT access token, httpOnly refresh cookie, CSRF double-submit, login audit.
RBAC model	Three-layer User → Role → Permission. Code checks permissions, not role names.
Data quality	Zero mock data. Dropdowns, lists, forms, dashboards, and auto-fill identity come from live MySQL through repository layer.
Audit doctrine	Every state-changing operation must have audit evidence. State machines and audit pairing are non-negotiable.

1.4 Phase completion dashboard

Phase	Name	Status	Locked Output
0	Discovery	DONE	23 clarifying questions resolved; 17 gaps surfaced; project reality captured.
1	Domain Modeling	DONE	5 personas, 3 critical journeys, 36+ business rules, 45 functional requirements.
2	Architecture	DONE	20 master decisions, 6 stack additions, 5 confirmations, 9-module map.
3	Database	RUNTIME READY	New tables, legacy alters, roles, permissions, migrations, seeds, verification checks.
4	Auth/RBAC	SEALED	HTTP + browser authentication, JWT/cookies, /me, middleware, protected routes.

Phase	Name	Status	Locked Output
5	Equipment	SEALED	First real feature: live list, types, create form, transaction, shell redesign.
6	Jobs Slice 1	SEALED REFERENCE	Job Requests + Job Cards doctrine, state machine design, row-level scope, three-screen slice.

1.5 The DS lock statement

DS LOCKED INTERPRETATION

Phase 0→6 is now the backbone memory. Future work must not casually re-decide these fundamentals. If an implementation detail must change, document it as an additive amendment with the reason, migration impact, and downstream modules affected.

Part II — Source Backbone Used for This Consolidation

The following uploaded documents were treated as the project backbone. This consolidated DOCX does not replace those source documents; it acts as a navigable senior-engineering synthesis that tells future developers what to remember and how to think.

Source Artifact	Role in Backbone	Observed Size
phase 0-----_3 F I N A L.docx	Phases 0→3 consolidated final documentation	753 paragraphs/table rows · ~78,073 chars
FINAL-DESC-CMCMIS.pdf	Canonical pre-build project blueprint	42 rendered pages · ~85,460 chars
PHASE3_COMPLETE_v2.0.md	Phase 3 complete runtime-ready migration bundle	901 lines · ~52,438 chars
phase4SEALED.docx	Phase 4 sealed Auth/RBAC build record	495 paragraphs/table rows · ~107,786 chars
phase5sealedLOCKED.docx	Phase 5 sealed Equipment module build record	521 paragraphs/table rows · ~106,791 chars
phase6FINALsealedLOCKED.docx	Phase 6 sealed Job Requests + Job Cards architecture reference	553 paragraphs/table rows · ~101,540 chars

2.1 Authority order

- **FINAL-DESC-CMCMIS controls behaviour.** It is the canonical pre-build blueprint for product behaviour, modules, roles, and rules.
- **Phase 3 v2.0 controls schema/runtime database design.** If behaviour and schema conflict, behaviour stays aligned to FINAL-DESC while schema changes are implemented through additive migrations.
- **Phase 4 controls auth implementation contracts.** req.user shape, cookies, middleware order, error envelope, protected route behaviour, and permission filtering must remain compatible.
- **Phase 5 controls the feature-module template.** Equipment shows the exact build pattern for future modules: validators → repo → service → controller → routes, plus FE hooks/pages/components.
- **Phase 6 controls job-flow doctrine.** For Job Requests and Job Cards, state machines, row-level scope, audit pairing, canonical names, and slice-based delivery are the lock.

2.2 Important environment naming note

DATABASE NAME NOTE

The uploaded phase documents repeatedly reference MySQL database `final` during sealed implementation. Earlier live project context also mentions `cmcmis\_redev`. The engineering rule is: never hard-code the DB name in module logic. DB_NAME must come from .env. The schema and repository contracts matter more than the local database alias.

Part III — Phase 0 → 6 Integrated Timeline

3.1 Phase 0 — Discovery insights

Phase 0 prevented a false start. Instead of coding immediately, the project surfaced unknowns around users, roles, equipment categories, authentication, deployment, PDF strategy, compliance, and the existing database.

Discovery Area	Locked Insight
Scope	Metrology, calibration, maintenance, repair, and registration for T&ME and F&PE equipment.
Users	Exactly five roles in v1; no separate Admin role. Admin responsibilities collapse into Super Admin.
Auth	Employee-ID login. Initial password equals employee_id at seed. Password pattern: two uppercase letters + five digits.
Deployment	Internal/on-premises; no public cloud; browser/API HTTPS is the only intended traffic path.
PDFs	Generated on demand and streamed; not persisted as files.
Data posture	Legacy data is preserved; new schema work must reconcile, not erase.

3.2 Phase 1 — Modeling insights

Phase 1 converted discovery into roles, personas, journeys, and business rules. This phase is where CMCMIS became a workflow system rather than a list of screens.

Persona	System Must Serve Them By
Super Admin	Role assignment, user activation/deactivation, master data ownership, system integrity, audit oversight.
Lab In-Charge	Approval queues, engineer assignment, completed-work verification, reopen/close actions, workload visibility.
Lab Engineer	Assigned job cards, task checklists, readings/observations, state-stepper actions, PDF generation.
Normal User	Raise requests, register equipment, track own requests, minimal-click flows.
View-Only User	Read-only auditor/management visibility without write affordances.

3.3 Phase 2 — Architecture insights

Phase 2 locked the shape of the system. The most important decision is not a library; it is the discipline that the backend is layered and that permissions, states, validation, and SQL each live in the right layer.

Decision Family	Locked Direction
Language	JavaScript + JSDoc + Zod. No TypeScript in v1.
Data access	Raw SQL with mysql2/promise and Repository Pattern. No ORM.
Backend structure	Routes → Controllers → Services → Repositories.
Frontend structure	Feature-based React/Vite/Tailwind folders.
Auth	JWT access 15 min + refresh cookie 7 days; CSRF protection on refresh.
PDF	pdfkit because it avoids Chromium/Puppeteer overhead in on-prem environments.
Logging	Pino structured logs.

Decision Family	Locked Direction
Deployment	Nginx reverse proxy in production; internal infrastructure.

3.4 Phase 3 — Database runtime-ready insights

Phase 3 is the first database-touching phase. Its victory is not just table creation; it is that the migration path became idempotent, auditable, bootstrap-safe, and test-verifiable.

Database Result	Insight
15 new MVP tables	users, roles, permissions, role_permissions, user_roles, refresh_tokens, login_audit, departments, sections, cmms_cont_mst, history/audit/export tables.
6 legacy tables altered	cmms_emp_mst, cmms equip_mst, cmms_jobrequest_mst, cmms_jobcard_mst, cmms_parameter_master, cmms_checklist_mst.
Legacy isolation	Unused/problematic legacy tables isolated with _legacy_ * strategy rather than dropped.
RBAC seed	5 roles, 40 atomic permissions, role-permission grants, two Super Admins SA79900 and AC77777.
Verification	Automated migration verification + E2E bootstrap/auth tests.

3.5 Phase 4 — Auth module sealed insights

Phase 4 gave the database an HTTP and browser layer. It shipped the login flow, token handling, middleware pipeline, /me endpoint, frontend auth context, protected routes, and permission-filtered navigation.

```
helmet → cors → compression → express.json → cookie-parser → pino-http
      → routers → authenticate → authorize → validate → controller
      → notFoundHandler → errorHandler
```

Auth Contract	Why It Matters
req.user shape	Every future module reads authenticated identity and permissions from one stable object.
Error envelope	All APIs return consistent { error: { code, message, details } } on failure.
Cookie strategy	Refresh token is httpOnly; CSRF token is readable for double-submit defence.
Permission filtering	Sidebar/routes render by permission strings, not role-name branching.
JavaScript-only lock	The frontend scaffold was rebuilt in JS when TypeScript examples conflicted with D1.

3.6 Phase 5 — Equipment module sealed insights

Phase 5 proved that Phase 4 was not just infrastructure: it could support a live feature. Equipment became the template for future feature modules.

Equipment Insight	Locked Meaning
Audit-first development	No repo code before real schema mapping. Every form field must resolve to a real table/column or stop and ask DS.
Zero mock data	List rows, dropdowns, submitter information, and form options come from MySQL.
Server-trusted submitter	The backend ignores any client-supplied submitted-by data and uses req.user.
Transactional create	Equipment row, accessories, T&C acceptance, and audit row must commit atomically.
Shell upgrade	Sidebar, TopBar, global search route surface, DataTable, Pagination, and form primitives

Equipment Insight	Locked Meaning
	became reusable.

3.7 Phase 6 — Jobs slice locked insights

Phase 6 defines the operational heart: Job Requests + Job Cards. Slice 1 is intentionally limited but architecturally deep: list requests, create/submit request, and list job cards with row-level scope.

Slice 1 Screen	Route	Primary Behaviour
Job Requests List	/job-requests	Read own/all depending on role and permission.
New Job Request	/job-requests/new	Create draft and submit with T&C and server-owned submitter snapshot.
Job Cards List	/job-cards	Read job cards filtered by role and assignment scope.

Part IV — Product & Domain Doctrine

4.1 Why this is not a CRUD application

A CRUD application stores rows. CMCMIS stores operational provenance. In a calibration and maintenance environment, the dangerous bug is not merely a failed save; it is a save that makes the laboratory unable to prove who approved, repaired, calibrated, verified, or exported a record.

Strategic Layer	What Lives Here	Failure Consequence
People	Roles, employees, assignments, permissions	Wrong access can corrupt records or break compliance.
Process	Equipment, Job Request, Job Card state machines	Wrong transitions create audit failures.
Assets	Instruments, serial numbers, locations, calibration due dates	Lost history weakens measurement provenance.
Money	Procurement, PO, spares, vendor/contract data	Bad chains create procurement audit issues.
Knowledge	Reports, PDFs, history timelines, audit trails	Institutional memory disappears.

4.2 Golden rule of module data flow

```

EQUIPMENT (source of truth)
↓
JOB REQUEST (intake / approval / assignment)
↓
JOB CARD (execution / readings / completion / verification)
↓
DOCUMENT/PDF (generated on demand)
↓
EQUIPMENT.last_calibration_date ← only controlled back-write

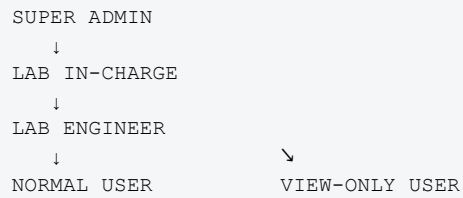
```

This rule protects auditability. Job Cards do not freely mutate Equipment. A single controlled hook updates calibration metadata after verified closure, and the state machine/audit trail surrounds that back-write.

4.3 The nine-module mental map

Module	Role in Product	MVP/Phase
Dashboard	Read-only operational visibility across modules.	MVP
Job Requests	Intake, draft, submit, approve/reject, assignment.	MVP / Phase 6+
Job Cards	Execution, checklist, readings, completion, verification, PDFs.	MVP / Phase 6+
Equipment	Asset master, registration, verification, calibration history.	MVP / Phase 5+
Schedule	Preventive maintenance + calibration planning.	Phase 2 / later
Procurement	Purchase orders, spares, warranty, vendor links.	Phase 2 / later
Inquiry	Search hub across vendors, products, job cards, instruments.	MVP visibility layer
Reports	Analytics, exports, utilization, calibration due, AMC, PO summaries.	Phase 2 / later
Admin	Users, roles, master data, lookup values, RBAC management.	RBAC in MVP; UI later

4.4 Role model in product language



The hierarchy is not a decorative org chart; it affects permissions, row visibility, workflow authority, and UI actions. View-Only is not below Normal User operationally; it is a read-only oversight path.

Part V — Architecture Doctrine

5.1 Runtime architecture

```

Browser (React + Vite + Tailwind)
  ↓ HTTPS / REST JSON
Nginx reverse proxy
  ↓
Node.js + Express API
  ↓ raw SQL / mysql2 promise pool
MySQL database (legacy + MVP tables + audit/history)

```

Layer	Contract
Frontend	Permission-aware routes, forms, table components, hooks, axios API wrappers, no direct DB logic.
Backend route/controller	HTTP shape only: request extraction, response status, next(error).
Backend service	Business rules, transactions, state transitions, audit pairing.
Backend repo	Only layer that knows SQL and real DB column names.
Database	Constraints, indexes, history tables, seeded RBAC, audit evidence.

5.2 Backend module build order

```

1. validators.js      → input contract
2. stateMachine.js    → legal transitions (where stateful)
3. repo.js            → SQL, canonical-to-real mapping, prepared statements
4. service.js         → business logic, transactions, audit pairing
5. controller.js      → HTTP shim
6. routes.js          → middleware order + URL binding

```

WHY BOTTOM-UP MATTERS

Routes-first creates fake productivity and hidden mocks. Bottom-up development makes every saved layer testable and keeps import arrows clean. This pattern is now inherited from Phase 5 and formalized in Phase 6.

5.3 Frontend architecture pattern

Frontend Area	Rule
AuthProvider	Owns auth hydration, token refresh, and user/permission state.
ProtectedRoute	Blocks anonymous users and renders Forbidden for missing permissions.
Layout	Wraps authenticated pages with Sidebar + TopBar shell.
Feature pages	pages/<module> contains list/form/detail placeholders.
lib/api/<module>.js	Thin axios wrappers only; no UI logic.
hooks	Own fetch/cancel/debounce/cache behaviour for lists.
schemas	Client zod mirrors backend validation where needed.

5.4 Single error-envelope law

```
{
```

```
"error": {  
  "code": "FORBIDDEN",  
  "message": "Insufficient permissions",  
  "details": null  
}
```

Every module must return errors through the centralized error handler. Feature modules must not invent one-off error shapes because the frontend is designed to consume the standard envelope.

Part VI — Database & Schema Insights

6.1 The database strategy in one sentence

DATABASE STRATEGY

Preserve legacy production data, add only what the MVP requires, isolate unused legacy tables safely, seed RBAC deterministically, and verify migrations automatically before any feature module depends on them.

6.2 New tables introduced by Phase 3

Table Family	Tables / Purpose
Identity/RBAC	users, roles, permissions, role_permissions, user_roles
Sessions/Auth	refresh_tokens, login_audit
Organisation	departments, sections
Vendor master	cmms_cont_mst
State history	equipment_status_history, job_request_status_history
Audit/export	audit_log, audit_log_changes, export_audit

6.3 The migration discipline

Discipline	Meaning
Idempotent	Re-running a migration does not corrupt or duplicate critical structures. CREATE IF NOT EXISTS, INSERT IGNORE, checksum table, existence checks.
Additive	Add columns, indexes, tables, and seeds; do not destructively rewrite sealed data.
Audited	Bootstrap users, role assignment, password set, and future state changes are recorded.
Verified	Automated checks after migrations confirm roles, permissions, users, bcrypt, and grants.
Traceable	M1–M12 answers map directly to migration files and defaults.

6.4 Role/permission seed insight

The RBAC seed is the bridge between business policy and code. Controllers and routes do not ask “is this user Lab In-Charge?” They ask “does this user have job_request:approve?” This means future policy changes happen in role_permissions, not by rewriting every controller.

Role	Permission Meaning
SUPER_ADMIN	All permissions and full governance authority.
LAB_IN_CHARGE	Approve/assign/verify/close lifecycle actions; not hard-delete/master-admin unless explicitly granted.
LAB_ENGINEER	Execute assigned jobs, update tasks/readings, generate PDFs, read necessary queues.
NORMAL_USER	Raise requests, register equipment, read own requests, dashboard/inquiry basics.
VIEW_ONLY	Read-only visibility and export where permitted; no state-changing operations.

6.5 Schema audit rule for every future module

Before writing repo.js:

```
SHOW TABLES / SHOW CREATE TABLE / SHOW INDEX
map every UI field to a real table+column
identify missing indexes before list endpoint code
ask DS before inventing schema
document canonical name → real column name mapping
```

6.6 Phase 6 schema additions to remember

Addition	Why It Exists
request_code / card_code	Human-readable JR-2026-NNNN and JC-2026-NNNN identifiers for lists, URLs, audit, and users.
submitter snapshot fields	Name/employee/designation/email captured at request creation so historical records survive user renames/transfers.
T&C evidence	tnc_accepted_at and tnc_version support legal/compliance traceability.
priority and status indexes	Fast list filtering and queue ordering.
code_counters	Atomic, year-scoped sequential code generation without unsafe locks.
status-history tables	Permanent transition timeline for future detail pages and audits.

Part VII — Auth/RBAC & Security Insights

7.1 Auth flow

```
POST /auth/login
  validate employee_id + password pattern
  find active/unlocked user
  bcrypt.compare(password, password_hash)
  load role + permissions
  issue access JWT
  set httpOnly refresh cookie + CSRF cookie
  write login_audit
  return user + permissions
```

7.2 Token/session envelopes

Envelope	Locked Behaviour
Access token	Short-lived JWT, roughly 15 minutes, held by frontend auth flow.
Refresh token	7-day refresh cookie, httpOnly, hashed in DB, rotated on refresh.
Idle/session protection	Session discipline enforced through token expiry/refresh flow; logout revokes refresh state.
CSRF defence	Double-submit token on refresh. FE reads CSRF cookie and sends header; backend compares.

7.3 Middleware order that future modules inherit

```
authenticate → authorize(permission) or authorizeAny(...) → rowLevelScope → validate(schema) → controller
```

Never put validation before authentication when validation depends on user context. Never put controller before authorization. Never allow a route to “temporarily” skip permission checks because that becomes production behaviour.

7.4 Security controls already wired

Threat	Control
SQL injection	Prepared statements, sort allow-lists, multipleStatements disabled at runtime.
Credential brute force	Login rate limit, failed_login_count, lockout policy, login_audit rows.
Token theft	Refresh token rotation, hashed refresh_tokens, theft detection via reused/revoked tokens.
XSS refresh theft	httpOnly refresh cookie prevents JavaScript access.
CSRF on refresh	CSRF token cookie + header double-submit.
Enumeration	Standardized auth failure messages and consistent error envelope.
Log leakage	Pino redaction of Authorization, Cookie, Set-Cookie, password/hash fields.

7.5 Row-level visibility rules

Role	Visibility Scope
Normal User	Own job requests only.
Lab Engineer	Assigned jobs plus relevant queue depending on module.

Role	Visibility Scope
Lab In-Charge	All jobs/equipment in operational scope; approval/verification authority.
Super Admin	All records and governance controls.
View-Only	Read all permitted data without state-changing actions.

Part VIII — Equipment Module Insights

8.1 Why Equipment was the first real feature

Equipment is the source of truth for the rest of the system. Job Requests reference equipment. Job Cards touch equipment. Calibration due dashboards depend on equipment. Inquiry and Reports read equipment. Therefore Phase 5 validated the entire feature-stack before the jobs workflow was built.

8.2 Equipment list contract

Feature	Implementation Insight
Pagination	Server-side LIMIT/OFFSET. Page size bounded.
Search	Debounced and abortable frontend calls; backend queries indexed fields.
Sorting	Allow-listed sort keys only to prevent SQL injection via ORDER BY.
Calibration color	Frontend utility maps next_cal_due_date into green/yellow/red badge logic.
Permissions	List/detail visible broadly; create/update/verify/condemn/delete gated separately.

8.3 Equipment create contract

```
POST /equipment
  authenticate
  authorize("equipment:create")
  validate(createEquipmentSchema)
  service opens transaction
  enforce T&ME/F&PE calibration rules
  insert equipment row with PENDING_VERIFICATION
  insert accessories if present
  store T&C acceptance JSON
  write audit_log
  commit
```

Rule	Why It Matters
PENDING_VERIFICATION default	Cross-role registration is safe because operational activation requires LIC/SA verification.
Server-owned submitter	Client cannot impersonate another employee in submitted-by fields.
Serial uniqueness	Prevents duplicate identity for instruments.
T&ME calibration frequency required	Test/measurement equipment must have calibration schedule metadata.
Transaction boundary	Equipment row, child rows, T&C, and audit must succeed/fail together.

8.4 Equipment lifecycle

```
PENDING_VERIFICATION → ACTIVE
ACTIVE → UNDER_CALIBRATION → ACTIVE
ACTIVE → UNDER_REPAIR → ACTIVE
ACTIVE/UNDER_* → OUT_OF_TOLERANCE / QUARANTINED / CONDEMNED / RETIRED
```

The exact allowed transitions should remain centralized in a state machine or service choke-point, never scattered across controllers.

8.5 Frontend shell contribution from Phase 5

- Sidebar now represents the 9-module CMCMIS navigation surface and filters by permission.
- TopBar adds global search route surface to `/inquiry?q=...`, preparing future Inquiry module work.
- DataTable and Pagination components are reusable for Job Requests, Job Cards, Reports, and Admin lists.
- EquipmentForm proves multi-section form architecture with react-hook-form and zod validation mirrors.

Part IX — Job Requests + Job Cards Insights

9.1 Why Phase 6 is the operational heart

Equipment is the noun; Jobs are the verbs. Phase 6 turns CMCMIS from a registry into a workflow system that captures requests, assignments, execution, verification, and future PDF evidence.

9.2 Job Request lifecycle

DRAFT → SUBMITTED → APPROVED → ASSIGNED → IN_PROGRESS → COMPLETED → VERIFIED_CLOSED
 ↘ REJECTED
 ↘ REOPENED (from closed/completed paths with reason)

Transition Moment	Control Needed
DRAFT → SUBMITTED	Submitter must be authenticated; submitted_by snapshot is server filled; T&C accepted.
SUBMITTED → APPROVED/REJECTED	Lab In-Charge/Super Admin permission; rejection reason mandatory.
APPROVED → ASSIGNED	Engineer selected; workload visibility and permission gate.
ASSIGNED → IN_PROGRESS	Assigned engineer starts work.
IN_PROGRESS → COMPLETED	Required readings/checklists complete.
COMPLETED → VERIFIED_CLOSED	Lab In-Charge verifies and closes.

9.3 Job Card lifecycle

ASSIGNED → IN_PROGRESS → COMPLETED → VERIFIED_CLOSED
 ↘ REOPENED (LIC/SA with mandatory reason)

Phase 6 Slice 1 lists Job Cards but does not yet implement all mutations. Slice 2/3 must preserve the same state-machine and audit-pairing doctrine rather than directly updating status fields.

9.4 Phase 6 eight doctrines

#	Doctrine	Future-Code Meaning
1	Bottom-up layering	Build validators, state machine, repo, service, controller, routes in dependency order.
2	Canonical names	Only repo knows real DB column names; upper layers use clean domain names.
3	Single choke-point	Every state change passes through one transition function.
4	Idempotent additive migrations	No destructive Phase 6 migrations. Re-runnable scripts.
5	Defence in depth	Client, server schema, service logic, repo scope, and DB constraints cooperate.
6	Audit pairing	One state change = one audit row in the same transaction.
7	Code generation without contention	Atomic code_counters pattern for JR/JC identifiers.
8	Thin controllers	Controllers translate HTTP only; business logic lives in services.

9.5 Row-level scope is SQL, not decoration

PHASE 6 ROW-SCOPE INSIGHT

When you read “Normal User sees own requests”, think WHERE submitted_by_user_id = req.user.userId. When you read “Lab Engineer sees assigned jobs plus queue”, think SQL scope fragments. UI hiding is helpful, but server-side WHERE clauses are the real control.

9.6 What Slice 1 intentionally does not ship

- Job Request detail view.
- Approve/reject mutations and UI.
- Automatic Job Card creation hook on approve.
- Observation entry and calibration certificate generation.
- Dashboard widgets for job counts.
- Inquiry integration and bulk exports.

These are deferred, not forgotten. Deferral is strategic because detail layouts and approval edge cases should be informed by live request data rather than guesses.

Part X — End-to-End Operational Walkthroughs

10.1 Login to permission-aware shell

1. User opens /login and enters employee_id/password.
2. Backend validates pattern, verifies bcrypt, loads role and permissions.
3. Access token and refresh/CSRF cookies are issued.
4. Frontend AuthProvider hydrates user and permissions, then routes to dashboard.
5. Sidebar renders only nav items whose permission codes exist in user.permissions.

10.2 Equipment registration to verification

6. Normal User/Lab Engineer/Lab In-Charge/Super Admin opens /equipment/new.
7. Form loads live types/divisions/accessory options from MySQL.
8. User completes equipment, procurement, complaint/contact, and T&C sections.
9. Backend creates equipment in PENDING_VERIFICATION using req.user as registered_by.
10. Lab In-Charge/Super Admin later verifies PENDING_VERIFICATION → ACTIVE.
11. equipment_status_history and audit_log preserve the transition trail.

10.3 Calibration job journey

12. Dashboard or list indicates equipment due/overdue for calibration.
13. User raises a Job Request of type CALIBRATION and submits it.
14. Lab In-Charge reviews submitted request, approves, and assigns engineer.
15. System creates/links a Job Card and updates equipment state to UNDER_CALIBRATION.
16. Engineer performs work, fills tasks/readings/observations, and marks complete.
17. Lab In-Charge verifies and closes, triggering controlled back-write to equipment calibration metadata.
18. PDF is generated on demand; export_audit records the export event.

10.4 Audit trace journey

19. View-Only or authorized role searches an instrument in Inquiry.
20. Equipment detail timeline shows status history, linked requests, linked job cards, and calibration history.
21. Audit logs show who changed what, when, from where, and with which before/after values.
22. Generated PDFs are not stored as files; the system recreates them from current/linked database state and logs export.

Part XI — Implementation Laws & Review Checklists

11.1 Non-negotiable project laws

Law	Review Question
JavaScript only	Did any .ts/.tsx, tsconfig, or @types dependency appear? If yes, reject.
No mock data	Does the UI read real MySQL data through API/repo? If no, reject.
Permission not role	Does code check permission strings rather than role names? If not, refactor.
Repo owns SQL	Does any controller/service contain raw SQL? If yes, move it to repo.
State machine owns transitions	Does any file directly UPDATE status outside the choke-point? If yes, reject.
Audit inside transaction	Can a state change commit without an audit row? If yes, reject.
Server-owned identity	Can client override submitted_by/registered_by fields? If yes, reject.
Idempotent migration	Can migration run twice safely? If no, rewrite.

11.2 Backend code review checklist

- Route order: authenticate → authorize/authorizeAny → rowLevelScope where needed → validate → controller.
- Controller is thin; no business conditionals except HTTP mapping.
- Service opens transactions for multi-write operations and passes the same connection to audit helper.
- Repo uses placeholders for values; sort/order identifiers are allow-listed.
- Zod schemas use .strict() to reject unknown fields.
- Every mutation writes audit evidence or has an explicit documented exception.
- Errors use AppError/errors factory and central handler, not res.status scattered everywhere.

11.3 Frontend code review checklist

- Route is wrapped in ProtectedRoute with the correct permission.
- Page uses Layout and follows the shell pattern.
- API calls go through lib/api-client.js wrappers; no fetch scattered in components.
- Forms use react-hook-form + zod resolver where validation matters.
- Submit buttons show loading state and prevent double-submit.
- UI hides unauthorized actions, but backend still enforces authorization.
- List pages support loading, empty, error, and success states.

11.4 Database review checklist

- Every required UI field is mapped to a real table/column before code starts.
- New columns/tables are additive and guarded by existence checks.
- Indexes exist for list filters, sorts, and row-level scope predicates.
- Legacy tables are preserved or isolated, not dropped casually.
- Seed data is deterministic and re-runnable.
- Status history tables are append-only.

Part XII — Phase 7+ Handoff & Next Locked Build Path

12.1 Immediate next engineering target

NEXT TARGET

Complete the remaining Job Request + Job Card slices without violating Phase 6 doctrine: detail pages, approve/reject, assignment, auto-create Job Card, work execution, observations/readings, verify-close, reopen, PDF generation, and dashboard/inquiry integration.

12.2 Recommended sequence

Sequence	Build Item	Why Now
1	Job Request detail page	Users need a full record view before approval actions.
2	Approve/reject service + UI	Moves submitted requests into operational state.
3	Assignment + auto Job Card creation	Connects request intake to execution.
4	Job Card detail + checklist/readings	Enables engineer execution.
5	Verify-close + controlled equipment back-write	Completes calibration/repair lifecycle.
6	PDF generation + export audit	Creates operational document outputs.
7	Dashboard widgets	Summarizes live operational queues.
8	Inquiry integration	Makes records searchable across modules.

12.3 Risk register for the next phase

Risk	Why It Can Hurt	Mitigation
Schema drift	Future code may assume columns that do not exist.	Run audit-first discovery and update canonical map before repo code.
Permission bypass	A route may ship without authorize().	Route checklist and integration tests for 403.
Audit gap	State mutation commits without evidence.	Audit-pairing smoke tests and same-transaction logging.
State scattering	Developers update status directly in service/repo.	State machine choke-point and grep review for UPDATE status.
UI-only security	Action hidden in UI but backend accepts direct call.	Backend authorization and row-level scope tests.
PDF scope creep	Complex template work delays core workflow.	Ship simple pdftk output first; polish after verification.
Performance decay	List endpoints slow as data grows.	Indexes for filters/sorts; EXPLAIN every main list query.

12.4 Deployment readiness principles

- No hard-coded secrets, database names, JWT secrets, or CORS origins.
- Run migrations against a copy of production data before touching staging/live.
- Keep backups outside the app and verify restore procedure separately.
- Use structured logs and correlation IDs for backend troubleshooting.

- Do not enable multipleStatements in runtime database pool.
- Keep dependency footprint minimal and documented.

Appendix A — Glossary

Term	Meaning
CMCMIS	Computerized Maintenance & Calibration Management Information System.
T&ME	Test & Measurement Equipment. Calibration metadata is mandatory.
F&PE	Fabrication & Production Equipment. Calibration metadata may be optional.
RBAC	Role-Based Access Control implemented as User → Role → Permission.
Permission code	Atomic resource:action string such as equipment:create or job_card:verify-close.
Choke-point	Single function/module through which a state change must pass.
Audit pairing	Writing the business state change and audit row in the same transaction.
Canonical name	Clean service-layer field name independent from legacy DB column names.
Idempotent migration	Migration that can safely run multiple times.
Row-level scope	Server-side WHERE conditions derived from the authenticated user/role/permission context.

Appendix B — Final Lock Manifest

Lock Item	Final Statement
Phase 0→6 context	Remembered as the current backbone for CMCMIS_SIMPLIFIED.
DS style	Structured, step-by-step, table-wise, diagram-rich, beginner-to-advanced explanations.
Engineering posture	Senior software developer + senior documentation engineer mode.
Codebase posture	JavaScript only, raw SQL, live MySQL, no mock data, no casual schema invention.
Security posture	Permission-first RBAC, JWT + refresh cookie, CSRF, audit logs, rate limits, safe errors.
Database posture	Preserve legacy, additive migrations, verification gates, no destructive shortcuts.
Workflow posture	State machines and audit pairing are the backbone of all operational modules.
Future posture	Phase 7+ must extend, not rewrite, the sealed doctrine.

FINAL SENIOR ENGINEERING NOTE

The project is now strong because the difficult thinking has been converted into doctrine: who can act, what state can change, which layer owns which responsibility, how evidence is recorded, and how future modules must inherit the same pattern. This document is the compact operational memory of that doctrine.